

3-Mavzu: Pythonda tarmoq dasturlashga kirish.

6-mashg'ulot. Pythonda GUI paketidan foydalanib chat dasturini tuzishni yakunlash

O'quv savollari:

1. TCP client–server dasturini GUI ko'rinishda ishlab chiqish.
2. Xabar almashish oqimini (connect/send/receive) tekshirish va sinovdan o'tkazish.
3. Xatoliklarni qayta ishlash, holatlarni (status/reconnect) GUI'da boshqarish.

Mashg'ulot maqsadi:

PyQt5 yordamida yaratilgan chat dasturini to'liq ishlaydigan holatga keltirish, TCP client–server o'rtasida xabar almashish oqimini sinovdan o'tkazish, xatoliklarni qayta ishlash va holatlarni GUI'da boshqarish.

1. TCP Client–Server Dasturini GUI Ko'rinishda Ishlab Chiqish

Serverning GUI asosidagi modeli

Avvalgi mashg'ulotda biz faqat **klient** uchun GUI yaratgan edik. Endi esa **server** uchun ham PyQt5 asosida grafik interfeys yaratiladi. Server GUI foydalanuvchiga ulanishlar, xabarlar va holatlarni kuzatish imkonini beradi.

```
import sys
import socket
import threading
from PyQt5.QtWidgets import QApplication, QWidget, QTextEdit, QVBoxLayout,
QPushButton, QLabel

class ChatServer(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TCP Chat Server")
        self.setGeometry(250, 150, 400, 400)

        self.layout = QVBoxLayout()
        self.status_label = QLabel("Server ishga tayyor emas.")
        self.text_area = QTextEdit()
        self.text_area.setReadOnly(True)
        self.start_button = QPushButton("Serverni ishga tushurish")

        self.layout.addWidget(self.status_label)
        self.layout.addWidget(self.text_area)
        self.layout.addWidget(self.start_button)
        self.setLayout(self.layout)

        self.start_button.clicked.connect(self.start_server)
```

```

def start_server(self):
    self.server_thread = threading.Thread(target=self.run_server)
    self.server_thread.start()
    self.status_label.setText("Server ishga tushdi.")
    self.start_button.setEnabled(False)

def run_server(self):
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(("127.0.0.1", 5000))
    server_socket.listen(5)
    self.text_area.append("Server ulanishni kutmoqda...")

    conn, addr = server_socket.accept()
    self.text_area.append(f"Yangi klient ulandi: {addr}")

    while True:
        data = conn.recv(1024)
        if not data:
            break
        message = data.decode("utf-8")
        self.text_area.append(f"Klient: {message}")
        conn.sendall(f"Serverdan: {message}".encode("utf-8"))

    conn.close()
    self.text_area.append("Klient ulanmagan.")

```

Izohlar:

- `threading.Thread` yordamida server fon rejimida ishlaydi (GUI muzlamaydi).
- `text_area` oynasi chat tarixini ko'rsatadi.
- Har bir klientdan kelgan xabar GUI'da aks etadi.

Klient GUI (takomillashtirilgan versiya)

Endi 5-mashg'ulotdagi klient kodini yanada to'liq shaklga keltiramiz: status ko'rsatkich, reconnect funksiyasi va holat boshqaruvi bilan.

```

import sys
import socket
from PyQt5.QtWidgets import QApplication, QWidget, QVBoxLayout, QTextEdit,
QLineEdit, QPushButton, QLabel, QHBoxLayout
from PyQt5.QtCore import QThread, pyqtSignal

class ClientThread(QThread):
    message_signal = pyqtSignal(str)
    status_signal = pyqtSignal(str)
    error_signal = pyqtSignal(str)

    def __init__(self, ip, port):
        super().__init__()

```

```

self.ip = ip
self.port = port
self.running = True
self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

def run(self):
    try:
        self.client_socket.connect((self.ip, self.port))
        self.status_signal.emit("Serverga ulandi.")
    except Exception as e:
        self.error_signal.emit(f"Ulanishda xatolik: {e}")
        return

    while self.running:
        try:
            data = self.client_socket.recv(1024)
            if not data:
                break
            self.message_signal.emit(data.decode('utf-8'))
        except:
            self.error_signal.emit("Aloqa uzildi.")
            break

def send_message(self, msg):
    try:
        self.client_socket.sendall(msg.encode('utf-8'))
    except Exception as e:
        self.error_signal.emit(f"Yuborishda xatolik: {e}")

def stop(self):
    self.running = False
    self.client_socket.close()

```

Klient GUI interfeysi

```

class ChatClient(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Chat Klient")
        self.setGeometry(350, 200, 400, 450)

        self.layout = QVBoxLayout()
        self.status_label = QLabel("Serverga ulanmagan.")
        self.chat_area = QTextEdit()
        self.chat_area.setReadOnly(True)

        self.input_box = QLineEdit()
        self.send_button = QPushButton("Yuborish")
        self.reconnect_button = QPushButton("Qayta ulan")

        hbox = QHBoxLayout()

```

```

hbox.addWidget(self.input_box)
hbox.addWidget(self.send_button)
hbox.addWidget(self.reconnect_button)

self.layout.addWidget(self.status_label)
self.layout.addWidget(self.chat_area)
self.layout.addLayout(hbox)
self.setLayout(self.layout)

self.send_button.clicked.connect(self.send_message)
self.reconnect_button.clicked.connect(self.reconnect)

self.connect_to_server()

def connect_to_server(self):
    self.client_thread = ClientThread("127.0.0.1", 5000)
    self.client_thread.message_signal.connect(self.display_message)
    self.client_thread.status_signal.connect(self.update_status)
    self.client_thread.error_signal.connect(self.display_error)
    self.client_thread.start()

def send_message(self):
    msg = self.input_box.text().strip()
    if msg:
        self.client_thread.send_message(msg)
        self.chat_area.append(f"Siz: {msg}")
        self.input_box.clear()

def reconnect(self):
    self.chat_area.append("Qayta ulanmoqda...")
    self.connect_to_server()

def display_message(self, msg):
    self.chat_area.append(msg)

def update_status(self, status):
    self.status_label.setText(status)

def display_error(self, error):
    self.chat_area.append(f"<span style='color:red;'>{error}</span>")
    self.status_label.setText("Aloqa yo'q.")

```

Dasturlarni ishga tushirish

Serverni ishga tushirish:

```
python server_gui.py
```

Klientni ishga tushirish:

```
python client_gui.py
```

Natija:

- Server GUI oynasi: “Server ishga tushdi, klient ulanmoqda...”
- Klient GUI oynasi: “Serverga ulandi.”
- Xabar almashish: foydalanuvchi matn yuboradi va serverdan javobni oladi.

2. Xabar Almashish Oqimini (connect/send/receive) Tekshirish va Sinovdan O‘tkazish

Sinov ssenariysi

Bosqich	Amal	Natija
1	Serverni ishga tushirish	Server “ulanishni kutmoqda” holatida
2	Klient ulanadi	Server “yangi klient ulandi” deb yozadi
3	Klient xabar yuboradi	Server “Klient: ...” deb ko‘rsatadi
4	Server javob beradi	Klient “Serverdan: ...” deb oladi
5	Ulanish uziladi	Klientda “Aloqa yo‘q.” holati chiqadi

3. Xatoliklarni Qayta Ishlash, Holatlarni (status/reconnect) GUI’da Boshqarish

Ulanish holatlarini boshqarish

Klient oynasida `status_label` orqali holat doimo yangilanadi:

- “Serverga ulandi.”
- “Aloqa yo‘q.”
- “Qayta ulanmoqda...”

Xatoliklarni qayta ishlash

Xatoliklar signal orqali GUI oynasida qizil rangda aks ettiriladi:

```
self.error_signal.emit("Server bilan aloqa uzildi.")
```

GUI’da:

```
self.chat_area.append("<span style='color:red;'>Server bilan aloqa uzildi.</span>")
```

Qayta ulan (Reconnect)

Agar tarmoq uzilib qolsa, foydalanuvchi “**Qayta ulan**” tugmasini bosib, avtomatik ravishda yangi ulanishni yaratadi.

Xulosa

O‘rganilgan mavzular	Tavsif
GUI asosidagi server va klient	PyQt5 yordamida to‘liq interaktiv chat tizimi
Xabar almashish oqimi	connect, send, receive jarayonlarini to‘liq test qilish
Signal-slot mexanizmi	GUI orqali real vaqt rejimida aloqa boshqarish

Holatlarni boshqarish	Status, reconnect, va xatoliklar GUT'da aks ettirildi
------------------------------	---